

Color Channels Extension to Canny's and similar Custom Edge Detector ASB

Aashutosh Singh Baghel
dept. of Computer Science and
Engineering
Samrat Ashok Technological Institute
Vidisha, India
aashutosh28cse004@satiengg.in

Nikhil Kushwah
dept. of Electrical Engineering
Samrat Ashok Technological Institute
Vidisha, India
nikhil27ee009@satiengg.in

Mahesh Kushwah
dept. of Electrical Engineering
Samrat Ashok Technological Institute
Vidisha, India
maheshkushwah236@gmail.com

Abstract—This paper studies extensions to existing algorithms, and also explores new ways to tackle edge /contour detection in images. Current methods involve conversion of image to a greyscale image for edge detection with optional gaussian blurring for noise removal[1][2]. These conventional methods often do not make use of separate color channels. In this paper, we have utilized extra color channels over canny, and have also proposed and tested improvements over our custom edge detection algorithm ASB (AtrociousSnifferofBorders). Comparisons between Canny[1] and ASB are also made. Experimental results on images will provide insight on strength and limitations of each algorithm. ASB has been updated to be a multipass algorithm; now producing two images: one based on sequential output, and second a product of both passes.

Keywords—computer vision, edge detection, image analysis, image filtering

I. INTRODUCTION

An image is a two dimensional array of values[5]. Each cell holding either 1 (greyscale) or 3 (BGR or CMY) values. These cell depths, are hereafter referred as color channels. The process of edge detection aims to find cells that lie on the boundaries of two or more objects in the image[1][5]. These edges may be identified from relatively sharp changes in channel values, which may arise from: difference in distance of objects from camera, irregularities in texture of surface, presence of two distinct objects in image. An edge detection approach does the job of identifying these edges within an image. It needs to be robust to counter noise within the image, and be fast to be used for real-time applications.

Common approaches to identification of pixel edges are: single-step filters (Sobel, Prewitt, Scharr), multi-step procedures(Canny), and Machine-Learning based approaches (CNNs, RNNs) [1][3][4][6].

ASB's first iteration was a single filter, based of using differences between total of all channels of the central pixel and the total of all channels for every adjacent pixel. After this, a rudimentary hand-picked threshold was used to select edge-pixels and non-edge-pixels. Current approach is multipass and gives two outputs: a binary image(sequential image), and also a variable intensity image(product of both passes). We will dwelve into each approach next.

II. BACKGROUND OF ASB V1

A) Pre-processing

First, a gaussian filter is applied to reduce noise over the image[5],[7], a 5x5 kernel gives generally good results at the processing power used. Guassian filter also helps smoothen harsh textured surfaces like wood and rusty iron.

B) Variance Calculation.

After pre-processing, the next step is variance calculation. "Variance" refers to the difference between the color channels of two images.

Function for each pixel in the image will be:

$$S_{(i,j)} = \sum_{c=0}^2 \sum_{m=l-1}^{l+1} \sum_{l=i-1}^{i+1} [K_{i,j,c} - K_{l,m,c}]$$

Here, K = Channel value at a given channel c of pixel [i,j]
S = Variance/Difference between the centre pixel K and all adjacent pixels.

For correcting pixels at boundaries/image boundaries, a correction factor is multiplied to the affected pixels. This correcting factor β is calculated as follows:

$$\beta = \frac{Z \sum_{x=1}^{L-1} \sum_{y=1}^{M-1} S_{[i,j]}}{LM}$$

Here, β is the correction factor to be multiplies, L and M be height and width of the image. And S is the variance of the pixel.

After the variance for each pixel has been calculated and corrected, a threshold is manually picked. For every pixel where variance>=threshold, the pixel is marked as an edge. This process is repeated over all pixels in the image. At the end, we are left with a matrix with cells containing either a 1 (edge) or 0 (no-edge). This has striking similarity to Sobel and Scharr[3][4], and performs just as same in terms of performance and detection rates.

III. CHANGES IN ASBv2

A) ASBv2

In the previous iteration, the variance was calculated and manual thresholding was used. This resulted in good results for some images, not so good for others (unaccounted changes in brightness/exposure). In ASBv2, the threshold was switched from manual value to the RMS of variances[5].

This offered better chances, since, the peaks in the variance (edges) were identified, but using RMS also adds a lot of noise to the final image. This was tackled in ASBv3. ASBv2 has next to none development.

IV. FAULTS IN ASBv1 AND ASBv2

A) Variance

The biggest fault in both iterations was summation of color channels before difference calculation. This meant that two pixels that have same intensity with distinct colors (sum of channels are close, but distribution s wildly different), were identified as non-different[8]. This meant that in images where colors change but intensities do not, edges are never detected. This is accounted for in ASBv3 with separete branching of each channel before compositing together.

B) Thresholding

Although RMS of variances provide great detection, but the skewed distribution in peaks can cause the RMS to include noisy pixels in the final image. This is also accounted for in ASBv3 by switching to a different approach to deciding thresholds (Standard deviation)

V. ASBv3

ASBv3 is a multi-step multi-output algorithm. It returns two images, a binary edge image, consisting of edge-yes/edge-no pixels, and a cleaned difference pixel. The binary edge image is usable in cases where absolute pixels are required (YOLO CNNs), while the second, more flexible difference image can be used in places where gradients are acceptable. (human analysis in Xrays, MRIs etc.)

A) Pass 1.

This involves pre-processing (gaussian blurring) and seperation of image into component channels. Three dummy matrices are formed (each filled with zeros), and over each, a simple diagonal **L1 norm-regularization** is applied[9].

$$P_{[i,j]} = (P_{[i-1,j-1]} + P_{[i+1,j+1]} + P_{[i-1,j+1]} + P_{[i+1,j-1]})$$

Here, $P_{[i,j]}$ is the cell in a given channel C of the three channels R,G,B or C,M,Y. This gives us three different matrices filled with variances. One matrix for each channel.

This step promotes sparsity in the image and is vital if we wish to multiply the results of this step with the results of the second pass. After these matrices have been obtained, All matrices are added back into a master matrix. This matrix is fundamentally different from ASBv1's variance matrix, as it contains summation of differences between channels, rather than difference between summation of channels. The output of pass 1 is this master matrix (**M**).

B) Pass 2

Pass 2 aims at filtration and thresholding. In ASBv3, we have calculated thresholds using standard deviation of the pixels[5] in the master matrix. This gives us a thresholded matrix with edge-pixels. But, this image needs to be converted into a binary image (only yes/no edge pixels). Before this step, the thresholded matrix is used to create a "clean image". This image has gradients and varying intensities, but is very effective in distinguishing noise from true edges. This "clean image" is created as follow:

$$C_{[i,j]} = (M_{[i,j]} * T_{[i,j]}) - M_{[i,j]}$$

Here,

C = "Cleaned" image

M = Master matrix (output of pass 1)

T = Thresholded matrix

This way, in pass 2, we are provided with 2 outputs, a variable intensity edge matrix, and a binary edge matrix.

C) The "Clean" image

- This is the product of pass1 and pass2's outputs.
- What makes it interesting is that since it is a product, the matrix is a product, the lower values of both matrices will multiply and non important textures/edges will supress themselves.
- The peaks will amplify due to being multiplied by other peaks, this results in higher edge confidence.
- The subtraction of the master matrix is being done to remove extra remaining noise. Values that go below 0 in subtraction are set to 0 to avoid integer overflow.

VI. CHALLENGES AND POSSIBLE FIXES

High computation costs remain one of the main concerns, since multiple passes require multiple operations per pixel and also high memory usage. Time completexity of ASBv3: $O(3*NM+4*NM+3*NM+NM+NM+NM)=O(k*NM)$. "k" is the number of steps in the algorithm.

Also, pepper and salt noise still remain s big problem with no solution in sight other than extensive Guassian blurring. This itself is a problem due to high performance tax of Gaussian blur.

A possible fix to the second problem can be utilization of codec of the image itself, by identifying blocks that are loosely packed/have high compression (will have lower edges chances).

VII. RESULTS

The order of provided images are:

- 1) Original Image
- 2) Master Matrix (normalized to uint8 0-255)
- 3) Threshold_image
- 4) Clean Image
- 5) Canny detection

I. Helmet Test

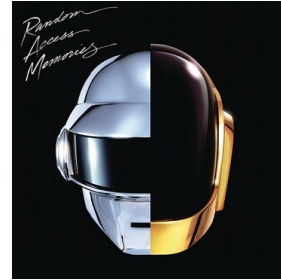


Figure 1.1: Helmet Original Image



Figure 1.2: Master Matrix normalized to 8 bit integer space

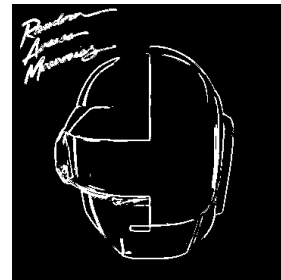


Figure 1.3: Binary thresholded image P2.A

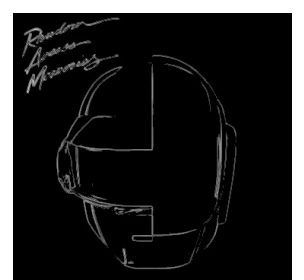


Figure 1.4: Cleaned variance image P2.B

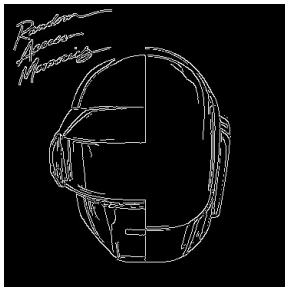


Figure 1.5: Canny's edges
 $LT=100, LH=200$

II. Person Test



Figure 2.1: Original Image of a person

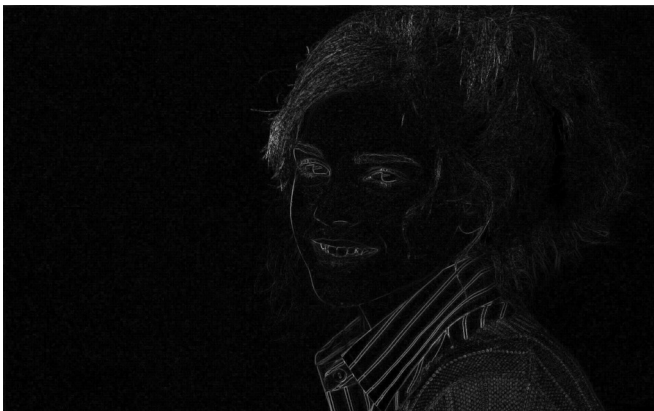


Figure 2.2: Master Matrix (pass 1) of the image



Figure 2.3: Thresholded image (pass 2 result A)



Figure 2.4: Cleaned Image (Product of threshold and master) (pass 2 result B)



Figure 1: Canny's edges $LT=100, HT=200$

III. Text and dark environment person test



Figure 3.1: Original text and light



Figure 3.2: Master matrix

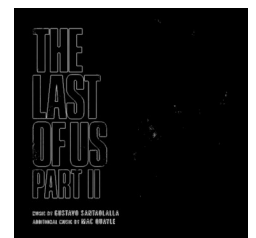


Figure 3.4: Cleaned image P2.B

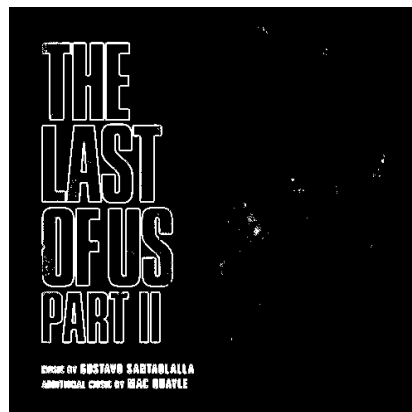


Figure 3.3: Thresholded image P2.A



Figure 3.5: Canny's edges $LT=100, HT=200$

ACKNOWLEDGMENT

I would like to express my sincere gratitude to Mr. Nikhil Kushwah for support throughout this project, especially in the implementation of the algorithm in the project. Their valuable insights and constructive feedback have been instrumental in shaping this project.

I am equally thankful to the developers of OpenCV, for their indispensable toolkit and software platform, without which, this work could not have been possible.

REFERENCES

- [1] J. Canny, "A Computational Approach to Edge Detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-8, no. 6, pp. 679–698, Nov. 1986.
- [2] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.
- [3] I. Sobel, "An Isotropic 3×3 Image Gradient Operator," *Stanford Artificial Intelligence Project (SAIL)*, 1968.
- [4] J. M. S. Prewitt, "Object Enhancement and Extraction," in *Picture Processing and Psychopictorics*, Academic Press, 1970.
- [5] R. C. Gonzalez, R. E. Woods, and S. L. Eddins, *Digital Image Processing Using MATLAB*, Pearson, 2004.
- [6] S. Xie and Z. Tu, "Holistically-Nested Edge Detection," in *Proc. IEEE International Conference on Computer Vision (ICCV)*, 2015.
- [7] D. Marr and E. Hildreth, "Theory of Edge Detection," *Proceedings of the Royal Society of London*, vol. 207, no. 1167, pp. 187–217, 1980.
- [8] T. Gevers and A. W. M. Smeulders, "Color-Based Object Recognition," *Pattern Recognition*, vol. 32, no. 3, pp. 453–464, 1999.
- [9] R. Tibshirani, "Regression Shrinkage and Selection via the Lasso," *Journal of the Royal Statistical Society*, vol. 58, no. 1, pp. 267–288, 1996.